

Ödev 17: Mart'tan Sağ Çıkan Bir Planlayıcı

Tarihleri doğru işleyen bir şey kurun, sonra dört `Math` tuzağını yeniden üretin.

Programınızın yazdirdığı her satır her çalıştırmada birebir aynı olmalı, ve makinenin hangi saat dilimine ayarlı olduğundan bağımsız olmalı. Bu, sabit bir `Clock`, sabit tarihler ve tohumlanmış bir üreteç demek. Değişen çıktı bu ödevi geçersiz kılar.

Birinci Bölüm: Aylık Faturalama

`Scheduler.java` dosyasını klasik biçimde oluşturun.

Bir müşteri **31 Ocak 2026** tarihinde kaydoluyor. O tarihi sabit bir `Clock` üzerinden okuyun, argümansız `LocalDate.now()` ile değil.

Sonraki dört fatura tarihini **iki biçimde** yazdırın:

1. **Naif biçimde**, bir önceki sonuca tekrar tekrar bir ay ekleyerek
2. **Sabitlenmiş biçimde**, her zaman özgün kayıt tarihinden hesaplayarak

İkisi ikinci aydan itibaren ayrışıyor.

Açıklayın

Şunları kapsayan bir yorum yazın:

- `plusMonths` metodunun 31 Ocak'a ne yaptığı ve o kısmın neden doğru olduğu
- Naif sürümün hatasının neden tek seferlik değil **kalıcı** olduğu
- Sabitlenmiş sürümün farklı olarak ne yaptığı, tek cümlede

İkinci Bölüm: Yaz Saatini Aşan Bir Toplantı

6 Mart 2026 Cuma günü New York saatiyle 09:00'a bir toplantı koyun, sonra onu iki biçimde üç gün ileri taşıyın:

- `plusDays(3)`
- `plusHours(72)`

İkisini de yazdırın, artı özgün ile `plusDays` sonucu arasında geçen gerçek saat sayısını. 72 olmayacak.

Sonra Londra'nın her iki an için ne gördüğünü gösterin.

Üç Durum Daha

1. **Belirsiz saat.** 1 Kasım 2026'da New York saatiyle 01:30 iki kez yaşanır. İkisini de ve aralarındaki farkı yazdırın.
2. **Var olmayan saat.** 8 Mart 2026'da 02:30 hiç yaşanmaz. `atZone` metodunun onunla ne yaptığını yazdırın.
3. **Kayan fark.** Londra ve New York farklı tarihlerde değişir. Bunun beş saatlik farkı sabit yazan biri için ne anlama geldiğini söyleyin.

Açıklayın

`plusDays` ile `plusHours` metotlarının neden farklılaştığını, her birinin hangi soruyu cevapladığını, ve tekrarlayan bir toplantı ile bir zaman aşımı için hangisini kullanacağınızı kapsayan bir yorum.

Üçüncü Bölüm: Dört Math Tuzağı

Her birini yeniden üretin ve açıklayın:

1. `Math.abs(Integer.MIN_VALUE)` ve yol açtığı **negatif kova indeksi**. `abs(hash) % buckets` ile `floorMod(hash, buckets)` sonuçlarını yan yana gösterin.
2. `-7 / 2` ile `Math.floorDiv(-7, 2)` karşılaştırması
3. `Math.round(2.5)` ile `Math.round(-2.5)` karşılaştırması
4. Sessiz taşma, ve `Math.addExact` metodunun onun yerine ne yaptığı

Birinci tuzakta dikkatli olun. Kova sayınızı hatanın gerçekten görüneceği şekilde seçin. Bazı değerler her iki biçimi de aynı sonuca götürüp hatayı tamamen gizler, ve hangisinin öyle olduğunu bulmak alıştırmamanın bir parçası.

Dördüncü Bölüm: Tekrarlanabilir Örnekleme ve Biçimlendirme

Bir listeden rastgele N ad seçen, **tohumu** parametre olarak alan bir metot yazın.

Aynı tohumla iki kez ve farklı bir tohumla bir kez çağırın. İlk ikisinin uyuştuğunu gösterin.

Tohumsuz bir üretecin bir teste ne yapacağını ve bu üreticileri asla ne için kullanmamanız gerektiğini bir yorumda yazın.

`formatted` kullanan küçük bir biçimlendirilmiş rapor ile bitirin:

- Açık bir `Locale` üzerinden biçimlendirilmiş bir tarih
- Sola yaslı bir etiket sütunu
- Binlik ayrıçlı bir sayı

`Locale` değerinin neden isteğe bağlı olmadığını bir yorumda söyleyin.

Kabul Kriterleri

- ☐ `java Scheduler.java` ile çalışıyor
- ☐ İki kez çalıştırmak birebir aynı çıktıyı üretiyor
- ☐ Sabit bir `Clock` kullanılıyor; hiçbir yerde çıplak `now()` yok
- ☐ Her iki faturalama takvimi de yazdırılıyor ve ikinci aydan itibaren ayrışıyor
- ☐ Bir yorum naif hatanın neden kalıcı olduğunu açıklıyor
- ☐ `plusDays(3)` ile `plusHours(72)` farklı sonuç verirken gösteriliyor
- ☐ Geçen gerçek saat yazdırılıyor ve 72 değil
- ☐ Belirsiz saat, bir saat arrayla iki kez gösteriliyor
- ☐ Var olmayan saatin ileri taşındığı gösteriliyor
- ☐ Dört Math tuzağı da görünür çıktıyla yeniden üretiliyor
- ☐ Birinci tuzağın kova sayısı negatif indeksi gerçekten açığa çıkarıyor
- ☐ Örneklem tohumlanmış, ve aynı tohumlu iki çalıştırma uyuyor
- ☐ `ofPattern` metoduna açık bir `Locale` veriliyor, nedenini söyleyen yorumla

İleri Seviye

Yaz saati geçişinde aynı **yerel** saatte kalan haftalık bir tekrar döndüren `List<ZonedDateTime> weeklyMeetings(ZonedDateTime first, int count)` yazın.

Sonra her tekrarı yedi günlük saniye ekleyerek hesaplanan bir `Instant` olarak saklasaydınız neyin ters gideceğini, ve bir yıl sonraki, dilim kuralları o zamana kadar değişebilecek bir toplantı için bunun yerine ne saklayacağınızı bir yorumda yazın.

İpucu, çıktınız çalıştırmalar arasında değişiyorsa

Argümansız `now()` çağrılarını ve tohumsuz oluşturulmuş üreteçleri arayın. Buradaki tek değişkenlik kaynağı bunlar, ve ikisinin de bağımlı oldukları şeyi parametre olarak alan bir biçimi var. Bu ödev için bir numara değil; zamana bağlı kodun genel olarak test edilebilir kılınma biçimi.